计算机图形学课程笔记

本笔记旨在为计算机图形学的学习者提供一份全面、系统且深入的参考资料。内容涵盖了从基础的数学知识到前沿的实时渲染技术，力求理论与实践相结合，帮助读者构建扎实的图形学知识体系。

第1章：计算机图形学概论

1.1 定义与核心问题

计算机图形学（Computer Graphics，CG）是研究如何利用计算机生成、处理和显示图形的学科。它融合了数学、物理、计算机科学和艺术，其核心任务可以概括为**建模 (Modeling)**、**渲染 (Rendering)** 和**动画 (Animation)**。

- **建模 (Modeling)**: 创建和表示三维场景中的物体几何形状和表面属性。
- **渲染 (Rendering)**: 根据虚拟摄像机、光照和材质信息，将三维场景转化为二维图像。
- **动画 (Animation)**: 为场景中的物体赋予随时间变化的动态效果。

图形学的核心问题始终围绕着如何以更真实、更高效、更可控的方式模拟现实世界。

1.2 应用领域

计算机图形学的应用无处不在，深刻地改变了我们的生活和工作方式。

领域	典型应用
娱乐	电子游戏、电影特效、动画片
设计	计算机辅助设计 (CAD)、建筑可视化、工业设计
科学与医学	数据可视化、医学成像 (CT, MRI)、科学模拟
虚拟与增强现实	VR/AR 应用、模拟训练
用户界面	图形用户界面 (GUI)、数据图表

1.3 发展历程

图形学的发展可以大致分为几个阶段：

1. **线框图时代**: 早期的图形学只能显示物体的线框结构。
2. **光栅化时代**: 随着硬件的发展，出现了能够填充多边形并进行简单光照计算的实时渲染管线。
3. **基于物理的渲染 (PBR)**: 追求更加真实感的渲染，引入了基于物理的光照和材质模型。

4. **实时光线追踪时代**: 以NVIDIA的RTX技术为代表，将电影级别的光线追踪技术带入实时渲染领域，实现了更加逼真的阴影、反射和全局光照效果。

第2章：数学基础\n\n数学是计算机图形学的基石。向量、矩阵和线性代数为我们提供了描述和操作三维空间中物体所需的工具。\n\n### 2.1 向量 (Vector)\n\n向量是具有大小和方向的量，在图形学中用于表示位置、位移、速度、方向等。\n\n- 向量表示: 一个三维向量可以表示为 $mathbfv=(x,y,z)$ 。\n- 向量的模 (Magnitude): 表示向量的长度，记作 $‖mathbfv‖=sqrtx2+y2+z2$ 。\n- 单位向量 (Unit Vector): 模为1的向量，用于表示方向。 $hatmathbfv=fracmathbfv|mathbfv|$ 。\n\n#### 2.1.1 向量点积 (Dot Product)\n\n点积是两个向量的乘积，结果是一个标量。它在光照计算中尤为重要。\n\n

$$mathbfacdotmathbfb=‖mathbfa‖‖mathbfb‖costheta=axbx+ayby+azbz$$

\n\n其中 $theta$ 是向量 $mathbfa$ 和 $mathbfb$ 之间的夹角。\n\n几何意义:\n- 可以用来计算两个向量之间的夹角。\n- 一个向量在另一个向量上的投影长度。\n- 如果 $mathbfacdotmathbfb>0$ ，则夹角小于90度；如果为0，则相互垂直；如果小于0，则夹角大于90度。\n\n#### 2.1.2 向量叉积 (Cross Product)\n\n叉积是两个向量的乘积，结果是一个新的向量，其方向垂直于原始两个向量构成的平面。\n\n

$$mathbfatimesmathbfb=(aybz-azby,azbx-axbz,axby-aybx)$$

\n\n几何意义:\n- 结果向量的方向遵循右手定则。\n- 结果向量的模等于以两个向量为边的平行四边形的面积: $\|mathbfatimesmathbfb\|=\|mathbfa\|\|mathbfb\|sintheta$ 。\n- 在图形学中，常用于计算一个平面的法线向量。\n\n### 2.2 矩阵 (Matrix)\n\n矩阵是组织成行和列的数字集合，是执行几何变换的核心工具。\n\n#### 2.2.1 矩阵乘法\n\n矩阵乘法不满足交换律，即 $ABneqBA$ 。一个 $mtimesn$ 的矩阵 A 乘以一个 $ntimesp$ 的矩阵 B 会得到一个 $mtimesp$ 的矩阵 C 。\n\n

$$Cij=sumk=1nAikBkj$$

\n\n#### 2.2.2 齐次坐标 (Homogeneous Coordinates)\n\n为了将平移、旋转和缩放等变换统一用矩阵乘法表示，我们引入了齐次坐标。通过增加一个额外的维度（w分量），我们可以用一个4x4的矩阵来表示三维空间中的所有仿射变换。\n\n- 点: 在齐次坐标中，一个三维点表示为 $(x,y,z,1)$ 。\n- 向量: 一个三维向量表示为 $(x,y,z,0)$ 。\n\n`w`分量的不同使得变换可以区分点和向量：平移变换只对点生效，而对向量无效。\n## 第3章：颜色与光照基础\n\n理解颜色和光照是创建逼真图像的关键。本章介绍基本的颜色模型和光照模型。

\n\n### 3.1 颜色模型 (Color Models)\n\n颜色模型是描述颜色的数学方式。\n\n- RGB模型: 一种加色模型，通过混合红（Red）、绿（Green）、蓝（Blue）三种基本颜色的光来产生各种颜色。这是计算机屏幕等显示设备最常用的模型。\n- CMYK模型: 一种减色模型，用于印刷行业。它通过从白光中减去青（Cyan）、品红（Magenta）、黄（Yellow）和黑（Key）来产生颜色。\n- HSV/HSB模型: 该模型更符合人类对颜色的直观感知，包含三个分量：色相（Hue）、饱和度（Saturation）和明度（Value/Brightness）。\n\n| 模型 | 主要用途 | 分量 | 特点 |\n| :--- | :--- | :--- | :--- |\n| RGB | 数字显示 | 红, 绿, 蓝 | 加色模型，硬件友好 |\n| CMYK | 印刷 | 青, 品红, 黄, 黑 | 减色模型，适用于油墨 |\n| HSV | 艺术创作, 图像编辑 | 色相, 饱和度, 明度 | 符合人类直觉 |\n\n### 3.2 基础光照模型\n\n光照模型用于计算当光线照射到物体表面时，我们能看到什么颜色。\n\n#### 3.2.1 环境光 (Ambient Lighting)\n\n即使在没有直接光源照射的情况下，场景中的物体也不会是完全黑暗的，这是因为光线会在环境中经过多次反弹。环境光就是对这种间接光照的简单模拟，它为整个场景提供一个基础亮度。\n\n

$$I_{ambient} = k_a \cdot I_{light}$$

\n\n- k_a: 材质的环境光反射系数。\n- I_{light}: 环境光的强度。\n\n#### 3.2.2 漫反射 (Diffuse Reflection)

\n\n漫反射模拟了光线从粗糙表面向各个方向均匀散射的效果。漫反射的强度取决于光线入射方向与表面法线的夹角，这符合兰伯特余弦定律 (Lambert's Cosine Law)。\n\n

$$I_{diffuse} = k_d \cdot (\mathbf{N} \cdot \mathbf{L})$$

\n\n- k_d : 材质的漫反射系数。\n- $\mathbf{N}$: 表面法线向量。\n- $\mathbf{L}$: 指向光源的向量。\n\n#### 3.2.3 镜面反射 (Specular Reflection)\n\n镜面反射模拟了光滑表面（如镜子、金属）上的高光效果。光线以与入射角相等的角度反射出去。\n\nPhong光照模型 使用反射向量 $\mathbf{R}$ 和视线向量 $\mathbf{V}$ 来计算镜面反射。\n\n

$$I_{specular} = k_s \cdot (\mathbf{R} \cdot \mathbf{V})^n$$

\n\n- k_s : 材质的镜面反射系数。\n- $\mathbf{R}$: 光线的反射向量, $\mathbf{R} = 2(\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L}$ 。\n- $\mathbf{V}$: 指向观察者的向量。\n- n : 高光指数（Shininess），控制高光的大小和锐利程度。\n\nBlinn-Phong光照模型 是对Phong模型的改进，它使用半程向量 (Halfway Vector) $\mathbf{H}$ ，计算更高效，并且在某些情况下效果更佳。\n\n

mathbfH=fracmathbfL+mathbfV|mathbfL+mathbfV|

Ispecular=kscdotIlightcdot(mathbfNcdotmathbfH)n

\n\n### 3.3 最终光照计算\n\n最终的颜色是环境光、漫反射和镜面反射三个分量的总和。\n\n

I=Iambient+Idiffuse+Ispecular

\n## 第4章：几何变换与投影\n\n几何变换和投影是渲染管线的核心，它们负责将三维模型从其局部空间转换到最终的二维屏幕上。这个过程主要通过一系列的矩阵变换来完成，统称为MVP变换。\n\n### 4.1 模型-视图-投影 (MVP) 变换\n\nMVP变换由三个主要的矩阵构成：模型矩阵 (Model Matrix)、视图矩阵 (View Matrix) 和投影矩阵 (Projection Matrix)。\n\n

Pscreen=MprojectioncdotMviewcdotMmodelcdotPlocal

\n\n#### 4.1.1 模型变换 (Model Transform)\n\n模型变换将模型的顶点坐标从其自身的局部坐标系（或称模型空间）转换到世界坐标系（World Space）。这允许我们在一个统一的场景中放置、旋转和缩放不同的模型。\n\n- 平移 (Translation): 将物体沿x, y, z轴移动。\n- 旋转 (Rotation): 将物体绕x, y, z轴或任意轴旋转。

\n- 缩放 (Scaling): 改变物体的大小。\n\n模型矩阵 $Mmodel$ 通常是平移、旋转和缩放矩阵的组合，例如 $Mmodel=TcdotRcdotS$。注意，变换的顺序非常重要，不同的顺序会产生完全不同的结果。\n\n#### 4.1.2 视图变换 (View Transform)\n\n视图变换将世界坐标系中的物体转换到观察坐标系（View Space 或 Eye Space）。这个坐标系是以摄像机为原点的，所有坐标都是相对于摄像机来定义的。\n\n我们可以想象成将整个世界移动和旋转，使得摄像机位于原点，并朝向-z轴方向。视图矩阵 $Mview$ 可以通过计算摄像机变换的逆矩阵得到。\n\n
\n
视图矩阵推导
\n假设摄像机位于点 $mathbfe$，观察方向为 $mathbfg$，上方向为 $mathbft$。我们可以构建一个以摄像机为中心的正交坐标系：\n\n1. 计算z轴方向（与观察方向相反）： $mathbfzc=-fracmathbfg|mathbfg|$。\n2. 计算x轴方向： $mathbfxc=fracmathbfttimesmathbfzc|mathbfttimesmathbfzc|$。\n3. 计算y轴方向： $mathbfyc=mathbfzctimesmathbfxc$。\n\n视图变换矩阵可以看作是先将世界坐标系原点平移到摄像机位置，然后再旋转到与摄像机坐标系对齐。\n\n

$$Mview=beginbmatrixxc,x&xc,y&xc,z&0yc,x&yc,y&yc,z&0zc,x&zc,y&zc,z&00&0&0&1endbmatrixcdotbeginbmatrix1&0&0&−ex0&1&0&$$

\n
\n\n### 4.2 投影变换 (Projection Transform)\n\n投影变换将观察空间中的三维坐标转换到裁剪空间 (Clip Space)，为最终的光栅化做准备。投影变换定义了一个称为视锥体 (View Frustum) 的可见空间，只有在视锥体内的物体才会被渲染。\n\n投影分为两种主要类型：正交投影和透视投影。\n\n#### 4.2.1 正交投影 (Orthographic Projection)\n\n正交投影创建了一个长方体的视锥体。它不进行透视缩放，因此物体的大小不会随距离而改变。适用于2D游戏、工程制图等场景。
\n\n
\n
正交投影矩阵推导
\n假设视锥体由六个平面定义：
$left,right,bottom,top,near,far$ 。\n我们的目标是将这个长方体 $[l,r]times[b,t]times[n,f]$ 映射到规范立方体 (Canonical View Volume) $[−1,1]3$ 。\n\n1. 平移: 将长方体中心移动到原点。\n2. 缩放: 将长方体的尺寸缩放到2x2x2。\n\n最终的正交投影矩阵为：\n\n

$$M_{ortho} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{n-f} & -\frac{n+f}{n-f} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

\n
\n\n#### 4.2.2 透视投影 (Perspective Projection)
\n\n透视投影模拟了人眼的视觉效果，即“近大远小”。它创建了一个金字塔形的视锥体。这是3D游戏和真实感渲染中最常用的投影方式。\n\n
\n
透视投影矩阵推导
\n透视投影的核心思想是：一个点离摄像机越远，它在投影平面上的坐标就越小。基于相似三角形原理，可以得到：\n\n

$$y' = \frac{n \cdot y}{z} \quad x' = \frac{n \cdot x}{z}$$

\n\n其中 n 是近平面距离。为了用矩阵表示这个除以 z 的操作，我们利用齐次坐标的 w 分量。变换后的点的 w 分量将等于其在观察空间中的 z 值。在后续的透视除法步骤中，所有分量都会除以 w 分量，从而实现透视效果。\n\n结合视场角 (FOV)、宽高比 (aspect)、近平面 (n) 和远平面 (f)，可以推导出透视投影矩阵：\n\n

$$M_{persp} = \begin{bmatrix} \frac{1}{\text{aspect} \cdot \tan(\frac{\text{fovY}}{2})} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\frac{\text{fovY}}{2})} & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

\n
\n\n### 4.3 视口变换 (Viewport Transform)\n\n经过MVP变换和透视除法后，坐标位于归一化设备坐标(NDC) 空间，通常是 $[-1,1]^3$ 的立方体。视口变换的最后一步是将这些NDC坐标映射到屏幕的像素坐标（视口）。\n\n

$$x_{screen} = \frac{x_{ndc} + 1}{2} \cdot \text{texttt{width}} + x_{viewport}$$

$$y_{screen} = \frac{y_{ndc} + 1}{2} \cdot \text{texttt{height}} + y_{viewport}$$

\n## 第5章：光栅化算法\n\n光栅化是将矢量图形（如点、线和三角形）转换为由像素组成的栅格图像（或称位图）的过程。这是实时渲染管线中的一个核心环节。\n\n### 5.1 直线光栅化\n\n直线光栅化的目标是在离散的像素网格上找到最佳的像素点来近似一条直线。\n\n#### 5.1.1 DDA算法 (Digital Differential Analyzer)\n\nDDA算法是一种基于增量的直线绘制算法。它通过在最大位移方向上以单位步长前进，并在另一方向上累加一个斜率增量来生成直线上的点。\n\n
\n
DDA算法实现
\n\n```cpp\nvoid DDALine(int x1, int y1, int x2, int y2) {\n int dx = x2 - x1;\n int dy = y2 - y1;\n int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);\n\n float xIncrement = dx / (float) steps;\n float yIncrement =
```
```

```
dy / (float) steps;\n\n float x = x1;\n float y = y1;\n for (int i = 0; i <= steps; i++) {\n drawPixel(round(x), round(y));\n x += xIncrement;\n y += yIncrement;\n }\n}\n
```
\n\n\n- 优点: 简单直观。\n- 缺点: 涉及到浮点数运算，效率较低，且存在累积误差。\n\n#### 5.1.2 Bresenham算法\n\nBresenham算法是一种更高效的直线绘制算法，它完全使用整数运算，避免了DDA算法中的浮点数问题。其核心思想是通过一个误差项来判断下一个像素点应该选择哪个位置。\n\n
\n
Bresenham算法推导与实现
\n假设我们正在绘制一条斜率在0到1之间的直线。对于当前点 (xk,yk)，下一个点的x坐标必定是 $xk+1=xk+1$。而y坐标要么是 yk，要么是 $yk+1$。\n\n我们定义一个决策参数 $pk=2Deltaycdotxk-2Deltaxcdotyk+c$。通过检查 pk 的符号，我们可以决定 y 是否需要增加。\n\n- 如果 $pk<0$，则选择点 $(xk+1,yk)$，并且 $pk+1=pk+2Deltay$。\n- 如果 $pkge0$，则选择点 $(xk+1,yk+1)$，并且 $pk+1=pk+2Deltay-2Deltax$。\n\n通过对不同象限的对称性处理，该算法可以扩展到所有斜率的直线。\n\n
```
cpp\nvoid BresenhamLine(int x1, int y1, int x2, int y2) {\n int dx = abs(x2 - x1);\n int dy = abs(y2 - y1);\n int sx = x1 < x2 ? 1 : -1;\n int sy = y1 < y2 ? 1 : -1;\n int err = dx - dy;\n\n while (true) {\n drawPixel(x1, y1);\n if (x1 == x2 && y1 == y2) break;\n int e2 = 2 * err;\n if (e2 > -dy) {\n err -= dy;\n x1 += sx;\n }\n if (e2 < dx) {\n err += dx;\n y1 += sy;\n }\n }\n}\n
```
\n

\n\n- 优点: 高效，只使用整数运算，精度高。\n- 缺点: 推导过程比DDA复杂。\n\n### 5.2 多边形光栅化 (扫描线填充)\n\n多边形光栅化（填充）最常用的算法是扫描线算法。其基本思想是按从上到下的顺序，用水平扫描线遍历多边形，计算每条扫描线与多边形边的交点，然后填充交点之间的像素。\n\n算法步骤:\n1. 建立边表 (Edge Table, ET): 将多边形的所有边按照其最小的y坐标放入一个桶中。每条边存储其最大y坐标、起始x坐标以及斜率的倒数 $1/m$。\n2. 初始化活动边表 (Active Edge Table, AET): AET是一个列表，用于存储当前扫描线所相交的边。\n3. 循环扫描: 从上到下逐行扫描。\n a. 将ET中对应当前扫描线的边移入AET中。\n b. 移除AET中已经到达最大y坐标的边。\n c. 对AET中的边按x坐标排序。\n d. 将AET中的边两两配对，填充它们之间的像素。\n e. 更新AET中每条边的x坐标： $x=x+1/m$。\n\n这种方法能够高效地填充任意形状的凸多边形和凹多边形。\n## 第6章：纹理映射技术\n\n纹理映射 (Texture Mapping) 是一种给三维模型表面添加细节、颜色和纹理的技术，它极大地增强了渲染的真实感，而无需增加模型的几何复杂度。\n\n### 6.1 UV坐标\n\n为了将一张二维的纹理图像“贴”到三维模型上，我们需要为模型的每个顶点指定纹理坐标，通常称为UV坐标。U坐标代表纹理的水平方向，V坐标代表垂直方向。UV坐标的范围通常在 $[0,1]$ 之间，其中 $(0,0)$ 通常对应纹理的左下角，$(1,1)$ 对应右上角。\n\n这个过程被称为UV展开 (UV Unwrapping)，即将三维模型的表面“展开”成一

个二维平面，以便与纹理图像对应。\n\n### 6.2 重心坐标插值 (Barycentric Interpolation)\n\n一旦我们为三角形的三个顶点指定了UV坐标，我们需要为三角形内部的每个像素（片元）计算其对应的UV坐标。这通过重心坐标插值来完成。\n\n对于三角形 $P1P2P3$ 内的任意一点 P，其位置可以表示为三个顶点的线性组合：\n\n

$$P=alphaP1+betaP2+gammaP3$$

\n\n其中，系数 $(alpha,beta,gamma)$ 就是点 P 的重心坐标，并且满足 $alpha+beta+gamma=1$。重心坐标可以通过面积比来计算。\n\n得到片元的重心坐标后，我们就可以用同样的系数来插值顶点的UV坐标，从而得到该片元在纹理上的精确位置。\n\n

$$uvpixel=alphauv1+betauv2+gammauv3$$

\n\n### 6.3 纹理采样与滤波\n\n当片元对应的UV坐标不是整数时，我们需要决定如何从纹理中采样颜色。这就是纹理滤波。\n\n#### 6.3.1 纹理放大 (Magnification)\n\n当纹理被放大，一个纹素 (texel) 可能覆盖多个像素。\n\n- 最近邻采样 (Nearest Neighbor): 选择最靠近UV坐标的那个纹素的颜色。速度快，但会导致块状（像素化）的外观。\n- 双线性插值 (Bilinear Interpolation): 取UV坐标周围最近的4个纹素

的颜色，并根据距离进行加权平均。结果更平滑，但计算成本稍高。\n\n#### 6.3.2 纹理缩小 (Minification) 与 Mipmapping\n\n当纹理被缩小时，一个像素可能覆盖多个纹素，如果处理不当会导致摩尔纹和闪烁。为了解决这个问题，我们使用 Mipmapping 技术。\n\nMipmap 是一系列预先计算好的、分辨率递减的纹理图像。渲染时，GPU会根据物体与摄像机的距离选择最合适的Mipmap层级进行采样。\n\n- 三线性插值 (Trilinear Interpolation): 在最接近的两个Mipmap层级上分别进行双线性插值，然后再对这两个结果进行线性插值。这提供了非常平滑的过渡效果。\n\n- 各向异性过滤 (Anisotropic Filtering, AF): 当物体表面与视线方向的夹角很大时（例如，从侧面看地面），传统的Mipmapping会导致纹理过度模糊。各向异性过滤通过在不同方向上采用不同数量的样本来解决这个问题，能够提供更清晰的图像，但计算成本也最高。\n\n| 滤波方法 | 质量 | 性能开销 | 主要应用场景 |\n| :--- | :--- | :--- | :--- |\n| 最近邻 | 低 | 非常低 | 像素艺术风格 |\n| 双线性 | 中 | 低 | 通用放大 |\n| 三线性 | 高 | 中 | 通用缩小，缓解闪烁 |\n| 各向异性 | 非常高 | 高 | 倾斜视角的表面 |\n## 第7章：深度缓冲与渲染管线\n\n本章将介绍用于解决可见性问题的关键技术——深度缓冲，并对现代图形渲染管线的整体流程进行梳理。\n\n### 7.1 深度缓冲 (Z-Buffer)\n\n在渲染三维场景时，我们需要确定哪些物体或物体的一部分是可见的，哪些是被其他物体遮挡的。这个问题被称为隐藏面消除 (Hidden Surface Removal)。

\n\nZ-Buffer（或称深度缓冲）是解决这个问题最常用的一种图像空间算法。\n\n核心思想:\n除了颜色缓冲（存储每个像素的颜色），我们还使用一个同样大小的深度缓冲（Depth Buffer）。深度缓冲存储了每个像素点上离摄像机最近的物体的深度值（通常是z坐标值）。\n\n算法流程:\n1. 初始化: 在渲染一帧开始时，将颜色缓冲区填充为背景色，并将深度缓冲区的所有值设置为无穷大（或最大值）。\n2. 光栅化: 对于场景中的每个三角形，当其被光栅化为片元时，计算每个片元的深度值。\n3. 深度测试 (Depth Test): 将当前片元的深度值与深度缓冲中对应位置的值进行比较。\n - 如果当前片元的深度值小于深度缓冲中的值，说明它离摄像机更近，是可见的。因此，用当前片元的颜色更新颜色缓冲区，并用当前片元的深度值更新深度缓冲区。\n - 如果当前片元的深度值大于或等于深度缓冲中的值，说明它被遮挡了，直接丢弃该片元。\n\n优点:\n- 简单高效，易于硬件实现。\n- 与物体的渲染顺序无关，可以并行处理。\n\n缺点:\n- 需要额外的显存来存储深度缓冲。\n- 对于透明物体，Z-Buffer无法正确处理，需要额外的技术（如从后向前排序渲染）。\n- 可能会出现Z-Fighting（深度冲突）问题：当两个物体的深度值非常接近时，由于浮点数的精度限制，可能会导致它们交错闪烁。这可以通过调整近/远平面距离或使用更高精度的深度缓冲来缓解。\n\n### 7.2 现代图形渲染管线\n\n现代GPU的渲染管线是一个高度流水线化和可编程的过程，它将三维数据转换为最终的二维图像。\n\n

\"渲染管线图\"

\n\n主要阶段:\n\n1. 应用阶段 (Application Stage) - CPU\n - 处理游戏逻辑、物理模拟、AI、用户输入等。\n - 准备场景数据（几何体、材质、光照、摄像机位置）。\n - 进行粗粒度的剔除（如视锥体剔除）。\n - 向GPU发送渲染命令 (Draw Calls)。\n\n2. 几何阶段 (Geometry Stage) - GPU\n - 顶点着色器 (Vertex Shader): 对每个顶点执行坐标变换（MVP变换），并处理其他逐顶点计算（如法线变换）。这是可编程的。\n - (可选) 曲面细分着色器 (Tessellation Shaders): 用于动态增加模型的几何细节。\n - (可选) 几何着色器 (Geometry Shader): 可以在GPU上创建、删除或修改图元（点、线、三角形）。\n - 图元装配 (Primitive Assembly): 将顶点组合成图元。\n - 裁剪 (Clipping): 裁剪掉完全在视锥体之外的图元。\n\n3. 光栅化阶段 (Rasterization Stage) - GPU (固定功能)\n - 将图元（如三角形）转换为一系列的片元（Fragment），即潜在的像素。\n - 为每个片元插值顶点属性（如UV坐标、颜色、法线）。\n\n4. 片元阶段 (Fragment Stage) - GPU\n - 片元着色器 (Fragment Shader / Pixel Shader): 计算每个片元的最终颜色。这是可编程的，负责处理纹理采样、光照计算等复杂逻辑。\n - 逐片元操作 (Per-Fragment Operations): 在片元颜色写入帧缓冲之前，进行一系列测试和操作，如深度测试、模板测试和颜色混合。\n\n5. 帧缓冲 (Framebuffer)\n - 最终的颜色、深度和模板值被写入帧缓冲中，等待显示到屏幕

上。\n## 第8章：高级光照模型\n\n为了实现更逼真的渲染效果，现代图形学已经从经验主义的Phong等模型转向了基于物理的渲染（Physically Based Rendering, PBR）。PBR旨在模拟光与物质在物理世界中的真实交互方式。\n\n### 8.1 基于物理的渲染 (PBR)\n\nPBR并非一个单一的算法，而是一套理论和方法的集合。其核心思想是使用基于物理原理的模型来描述材质和光照，并遵循能量守恒定律。\n\nPBR的三大核心概念:\n\n1. 微表面模型 (Microfacet Model): 假设物体表面在微观尺度上是粗糙的，由许多微小的、肉眼看不见的镜面构成。表面的粗糙度决定了这些微表面的朝向分布。\n2. 能量守恒 (Energy Conservation): 物体表面反射的光量不能超过其接收到的光量。一个更具反射性的表面，其漫反射部分就会更弱。\n3. 菲涅尔效应 (Fresnel Effect): 当光线以掠射角度（接近90度）照射到表面时，反射率会增加。几乎所有表面在掠射角度下都会呈现出近乎完美的镜面反射。\n\n### 8.2 BRDF 与渲染方程\n\n双向反射分布函数 (Bidirectional Reflectance Distribution Function, BRDF) 是PBR的核心。它是一个函数 $f(l,v)$ ，描述了给定入射光方向 l 和观察方向 v 时，表面反射的光线分布情况。\n\n所有BRDF都必须遵循能量守恒。将BRDF在整个半球上进行积分，即可得到渲染方程 (Rendering Equation) 的简化形式：\n\n

$$Lo(p,v)=intOmegaf(p,l,v)Li(p,l)(ncdotl)dl$$

\n\n- $Lo(p,v)$: 从点p沿方向v出射的光的辐射率。\n- $Li(p,l)$: 从方向l入射到点p的光的辐射率。\n- n : 表面法线。\n\n#### 8.2.1 Cook-Torrance BRDF\n\nCook-Torrance模型是实时渲染中最流行的PBR BRDF模型。它将BRDF分为漫反射和镜面反射两个部分：\n\n

$$fbrdf=kdflambert+ksfcook−torrance$$

\n\n- $flambert$ 是漫反射部分，通常采用Lambertian模型。\n- $fcook−torrance$ 是镜面反射部分，其表达式为：\n\n

$$fcook−torrance=fracDGF4(ncdotv)(ncdotl)$$

\n\n
\n
Cook-Torrance模型各分量详解
\n\n- D - 法线分布函数 (Normal Distribution Function): 估算微表面法线与半程向量 h 方向一致的概率。常用的有Trowbridge-Reitz GGX分布。粗糙度越低，法线越集中，高光越锐利。\n- G - 几何函数 (Geometry Function): 描述微表面之间的自遮蔽效应（光线被其他微表面遮挡）。常用的有Schlick-GGX近似。\n- F - 菲涅尔方程 (Fresnel Equation): 描述在不同入射角度下，光线被反射的比例。常用的有Schlick近

似，它使用基础反射率 $F0$ 进行插值。\n- 分母: $4(ncdotv)(ncdotl)$ 是一个归一化因子，用于确保能量守恒。\n\n
\n\n### 8.3 基于图像的光照 (Image-Based Lighting, IBL)\n\n渲染方程需要对场景中所有方向的入射光进行积分，这在实时渲染中是不现实的。IBL是一种通过采样环境贴图（Environment Map）来近似求解渲染方程的技术。\n\nIBL流程:\n1. 环境捕捉: 使用立方体贴图 (Cube Map) 或球谐函数 (Spherical Harmonics) 捕捉场景周围的环境光。\n2. 预计算/卷积: \n - 漫反射部分: 对环境贴图进行卷积，生成一个辐照度图 (Irradiance Map)。对于任意表面法线方向，可以直接从辐照度图中采样，获得该点的总漫反射光照。\n - 镜面反射部分: 预滤波环境贴图。生成不同粗糙度级别的Mipmap链，每个级别代表不同粗糙度下的预积分镜面反射结果。同时，预计算一个BRDF积分查找表（LUT），用于存储菲涅尔项在不同视角和粗糙度下的响应。\n3. 实时渲染: 在片元着色器中，根据表面的法线、粗糙度和反射率，从辐照度图、预滤波的环境贴图和BRDF查找表中采样，组合得到最终的IBL光照效果。\n\nIBL通过大量的预计算，将复杂的积分过程转变成了几次纹理采样，从而实现了高质量的实时全局光照效果。\n## 第9章：阴影算法\n\n阴影是增强三维场景真实感和空间感的关键元素。本章介绍几种主流的阴影生成算法。\n\n### 9.1 阴影映射 (Shadow Mapping)\n\n阴影映射是实时渲染中最常用的阴影技术。它是一种基于图像空间的算法，通

过从光源的视角渲染场景来生成阴影。\n\n算法流程:\n1. 深度遍（Depth Pass）: 将摄像机放置在光源位置，并朝向光源的照射方向。渲染整个场景，但只将每个片元的深度值（离光源的距离）写入一张称为阴影贴图 (Shadow Map) 的纹理中。\n2. 渲染遍（Shading Pass）: 正常地从主摄像机的视角渲染场景。在片元着色器中，对于每个片元，将其坐标变换到光源的裁剪空间。\n3. 深度比较: 用变换后的坐标（作为UV坐标）去查询阴影贴图，得到该位置离光源最近的物体的深度值 $D_{shadowmap}$。然后，将当前片元自身的深度值 $D_{current}$ 与 $D_{shadowmap}$ 进行比较。\n - 如果 $D_{current} > D_{shadowmap}$，说明当前片元位于另一个物体之后，即处于阴影中。\n - 如果 $D_{current} \le D_{shadowmap}$，说明当前片元是可见的，不受阴影影响。\n\n问题与解决方案:\n- 阴影锯齿 (Shadow Aliasing): 阴影贴图的分辨率是有限的，这会导致阴影边缘出现锯齿。可以通过增加阴影贴图分辨率或使用滤波技术来改善。\n- 透视走样 (Perspective Aliasing): 当光源视锥体与摄像机视锥体不匹配时，会导致近处的阴影分辨率过高，远处的过低。可以使用级联阴影贴图 (Cascaded Shadow Maps, CSM) 来缓解，即为摄像机视锥体的不同远近区域使用不同的阴影贴图。\n- 阴影粉刺 (Shadow Acne) 和彼得平移 (Peter Panning): 由于阴影贴图的精度问题和采样误差，可能会导致物体表面出现错误的自阴影条纹（Acne），或者阴影与物体分离（Peter Panning）。这通常通过在深度比较时添加一个

小的深度偏移 (Depth Bias) 来解决。\n\n### 9.2 软阴影 (Soft Shadows)\n\n现实世界中的阴影边缘通常是模糊的，因为光源具有一定的体积。阴影映射产生的硬边阴影看起来不真实。软阴影技术旨在模拟这种半影效果。\n\n#### 9.2.1 PCF (Percentage-Closer Filtering)\n\nPCF是实现软阴影最基本的方法。它不是只采样阴影贴图上的一个点，而是在当前片元周围的一个小区域内进行多次采样，然后计算落在阴影中的样本比例，以此作为阴影的强度。\n\n- 优点: 简单有效，可以平滑阴影边缘。\n- 缺点: 采样区域的大小是固定的，产生的半影宽度也是均匀的，这不符合物理规律。多次采样会增加性能开销。\n\n#### 9.2.2 PCSS (Percentage-Closer Soft Shadows)\n\nPCSS是一种更高级的软阴影技术，它模拟了阴影半影宽度随遮挡物与接收物之间距离变化的物理现象。\n\n算法流程:\n1. 遮挡物搜索 (Blocker Search): 在阴影贴图上的一定区域内搜索遮挡物，并计算它们的平均深度。\n2. 半影估算 (Penumbra Estimation): 根据遮挡物的平均深度和光源大小，估算出半影区域的大小。\n3. PCF滤波: 使用上一步计算出的半影大小作为PCF的滤波核大小，进行最终的阴影计算。\n\nPCSS能够产生随距离变化的、更符合物理规律的软阴影，但计算成本也更高。\n\n### 9.3 阴影体积 (Shadow Volumes)\n\n阴影体积是一种基于几何的阴影算法，它能够产生像素级精确的硬阴影。\n\n算法流程:\n1. 轮廓提取: 找到场景中所有物体朝向光源和背向光源的面的分界线，即轮廓边 (Silhouette

Edges)。\n2. 构建阴影体: 从光源位置出发，将轮廓边向远处拉伸，形成一个封闭的“阴影体积”。\n3. 模板缓冲测试: 渲染场景。当摄像机视线进入阴影体时，增加模板缓冲区的值；穿出时，则减少。最后，模板缓冲区中值不为0的像素就处于阴影中。\n\n- 优点: 能够产生非常锐利、精确的阴影，不受分辨率限制。\n- 缺点: 算法复杂，对几何体的拓扑结构要求高（必须是封闭的流形）。大量的填充操作会消耗大量的GPU填充率，性能开销很大。在现代渲染中已较少使用。\n## 第10章：曲线与曲面\n\n在计算机图形学中，除了使用多边形网格来表示模型外，我们还经常使用参数化的曲线和曲面来定义光滑的形状，这在CAD、工业设计和动画中尤为重要。\n\n### 10.1 参数曲线\n\n参数曲线使用一个参数（通常是 t）来定义空间中的点。\n\n

$$P(t) = (x(t), y(t), z(t))$$

\n\n#### 10.1.1 贝塞尔曲线 (Bézier Curves)\n\n贝塞尔曲线由一组控制点定义，曲线会平滑地穿过或靠近这些控制点。曲线的阶数由控制点的数量决定（n个控制点定义了n-1阶曲线）。\n\n一条n阶贝塞尔曲线可以由伯恩斯坦多项式 (Bernstein Polynomials) 定义：\n\n

$$P(t) = \sum_{i=0}^{n} B_{i,n}(t) P_i, \quad t \in [0,1]$$

\n\n其中 P_i 是控制点，$B_{i,n}(t) = C(n,i)(1-t)^{n-i}t^i$ 是伯恩斯坦基函数。\n\nde Casteljau算法 提供了一种几何直观且数值稳定的方法来计算贝塞尔曲线上的点。它通过对控制点进行递归的线性插值来找到最终的点。\n\n性质:\n- 曲线必然通过起始和结束控制点。\n- 曲线完全包含在其控制点的凸包内。\n- 不具备局部控制性，移动一个控制点会影响整条曲线的形状。\n\n#### 10.1.2 B样条曲线 (B-Splines)\n\nB样条（Basis Spline）曲线克服了贝塞尔曲线缺乏局部控制性的缺点。它也是由一组控制点定义的，但其形状由节点矢量 (Knot Vector) 控制。\n\n节点矢量是一系列非递减的参数值，它决定了基函数在哪个参数区间内是有效的。通过修改节点矢量，可以改变曲线的形状和连续性。\n\n性质:\n- 局部控制性: 移动一个控制点只会影响曲线的局部区域。\n- 灵活性: 可以在不增加控制点数量的情况下提高曲线的阶数，或者通过插入节点来增加局部控制的灵活性。\n\n#### 10.1.3 NURBS (Non-Uniform Rational B-Splines)\n\nNURBS是在B样条基础上的扩展，它通过为每个控制点引入一个权重 (Weight)，并使用有理函数（分子分母都是多项式）来定义基函数。\n\n- 优点: NURBS功能强大，不仅可以精确地表示自由形态的曲线，还可以精确地表示圆、椭圆等二次曲线，这是贝塞尔和B样条曲线做不到的。它已成为工业设计领域的标准。\n\n### 10.2 参数曲面\n\n参数曲面使用两个参数（通常是 u 和 v）来定义空间中的点，可以看作是参数曲线在另一个维度上的扩展。\n\n

$$P(u,v)=(x(u,v),y(u,v),z(u,v))$$

\n\n贝塞尔曲面、B样条曲面和NURBS曲面都可以通过在两个方向上应用相应曲线的张量积来构建。\n\n### 10.3 细分曲面 (Subdivision Surfaces)\n\n细分曲面是另一种创建光滑模型的方法。它从一个粗糙的多边形控制网格开始，通过递归地应用一套细分规则来生成更平滑、更精细的网格。\n\n算法流程:\n1. 拓扑细分: 在网格中插入新的顶点来增加面数。例如，在每条边的中点插入一个新顶点，并将每个四边形分割成四个更小的四边形。\n2. 几何平滑: 根据周围旧顶点的位置，计算所有顶点（包括旧顶点和新顶点）的新位置。这个过程就像对网格进行“平滑”或“平均化”。\n\n重复这个过程，网格会逐渐收敛到一个光滑的极限曲面。\n\n常用算法:\n- Catmull-Clark: 适用于任意多边形网格（主要是四边形），是动画行业中最常用的细分算法。\n- Loop: 适用于三角形网格。\n\n优点:\n- 可以从任意拓扑的网格开始，非常灵活。\n- 可以自然地产生不同细节层次（LOD）的模型。\n## 第11章：现代图形学技术\n\n随着GPU硬件的飞速发展，现代图形学技术正在向着更高的真实感、更强的性能和更大的灵活性演进。\n\n### 11.1 现代GPU架构\n\n现代GPU是高度并行的处理器，专为处理图形和数据并行计算任务而设计。其核心是流式多处理器 (Streaming Multiprocessor, SM)。\n\n- SIMT (Single Instruction, Multiple Threads): SM的核心工作模式。成千上万的线程以32或

64个为一组（称为Warp或Wavefront）的形式被调度，共同执行相同的指令，但处理不同的数据。\n- 专用硬件单元: 现代GPU集成了用于特定任务的硬件单元，以加速渲染过程，例如：\n - 光栅化器 (Rasterizer) 和 纹理单元 (Texture Units)\n - RT核心 (RT Cores): 用于加速光线追踪中的BVH（层次包围盒）遍历和光线-三角形相交测试。\n - Tensor核心 (Tensor Cores): 用于深度学习计算，在图形学中被广泛用于降噪（如NVIDIA的DLSS）和AI辅助创作。\n- 内存层次结构: GPU拥有复杂的内存系统，包括高速的片上共享内存/缓存和高带宽的显存（VRAM），旨在为大规模并行计算提供足够的数据吞吐量。\n\n### 11.2 可编程着色器 (Programmable Shaders)\n\n可编程着色器是现代渲染管线的核心，它赋予了开发者极大的灵活性来控制渲染的每一个细节。\n\n- 统一着色器架构: 现代GPU采用统一着色器架构，即GPU中的计算单元可以执行任何类型的着色器程序（顶点、片元、几何等），提高了硬件利用率。\n- 计算着色器 (Compute Shader): 这是一种通用的着-色器，不属于传统的图形管线。它可以利用GPU的强大计算能力来执行任意的并行计算任务，如物理模拟、图像处理、后处理效果、剔除等。\n\n### 11.3 显式图形API (Explicit Graphics APIs)\n\n传统的图形API（如OpenGL、DirectX 11）为开发者提供了高级的抽象，隐藏了底层硬件的复杂性，但同时也限制了性能优化的空间。为了充分挖掘现代GPU的潜力，Vulkan和DirectX 12等新一代显式API应运而生。\n\n核心特性:

\n- 更低的CPU开销: 显式API将大量原本由驱动程序在“幕后”完成的工作（如内存管理、状态校验）交给了应用程序开发者。这减少了驱动的开销，使得CPU可以更高效地向GPU提交渲染命令。\n- 多线程渲染: 显式API从设计之初就考虑了多线程。开发者可以在多个CPU核心上并行地构建和提交渲染命令列表，极大地提高了渲染吞吐量。\n- 管线状态对象 (Pipeline State Object, PSO): 将渲染管线的状态（如着色器、混合状态、深度测试状态等）预先编译并打包成一个对象。这使得GPU可以非常快速地切换渲染状态，减少了Draw Call之间的性能瓶颈。\n- 显式控制: 开发者需要显式地管理内存分配、资源同步（使用Fence和Semaphore）和命令队列，这虽然增加了编程的复杂度，但也提供了前所未有的控制力和性能潜力。\n\n| 特性 | 传统API (OpenGL/DX11) | 显式API (Vulkan/DX12) |\n| :--- | :--- | :--- |\n| CPU开销 | 较高 | 低 |\n| 多线程 | 有限/困难 | 原生支持 |\n| 状态管理 | 隐式状态机 | 管线状态对象 (PSO) |\n| 控制力 | 低（驱动主导） | 高（应用主导） |\n| 易用性 | 较高 | 较低 |\n\n### 11.4 实时光线追踪 (Real-Time Ray Tracing)\n\n实时光线追踪是近年来图形学领域最重要的突破。通过专用的RT核心，GPU现在可以在实时帧率下追踪数百万条光线，从而实现比传统光栅化更逼真的效果。\n\n- 混合渲染 (Hybrid Rendering): 目前主流的应用方式是混合渲染，即以高效的光栅化为主，辅以光线追踪来处理特定的效果，如：\n - 精确的阴影: 产生物理正确的软阴影。\n - 逼真的反射和折射: 渲染镜面、水

面等材质。\n - 全局光照 (Global Illumination): 模拟光线在场景中的多次反弹，实现更真实的环境光照。\n- AI降噪 (AI Denoising): 由于实时追踪的光线数量有限，结果会充满噪声。AI降噪（如DLSS）利用Tensor核心来智能地去除这些噪声，从而在保证性能的同时获得高质量的图像。\n## 第12章：实时渲染优化\n\n在实时渲染（如电子游戏）中，维持稳定且高帧率至关重要。渲染优化的目标是在保证可接受的视觉质量的前提下，尽可能地减少GPU和CPU的负载。\n\n优化的核心思想是：“不要做无效的工作”。即，只渲染那些看得到且重要的东西。\n\n### 12.1 剔除技术 (Culling)\n\n剔除技术旨在从渲染管线的早期阶段就移除那些最终对画面没有贡献的物体或多边形。\n\n- 视锥体剔除 (View Frustum Culling): 最基本的剔除技术。在CPU端，检查物体的包围盒（Bounding Box）是否完全位于摄像机的视锥体之外。如果是，则该物体及其所有多边形都不会被发送到GPU。\n- 背面剔除 (Back-face Culling): 在几何阶段，检查三角形的法线方向。如果法线背向摄像机（即我们看到的是三角形的背面），则该三角形会被剔除。这对于封闭的实体模型非常有效，几乎可以剔除一半的多边形。\n- 遮挡剔除 (Occlusion Culling): 一种更高级的剔除技术，用于剔除那些被其他不透明物体完全遮挡的物体。实现方式多样，例如：\n - 硬件遮挡查询 (Hardware Occlusion Queries): GPU提供的一种机制，可以查询一个物体是否有任何像素通过了深度测试。\n - 层次Z缓冲 (Hierarchical-Z Buffer, Hi-Z): 利用

深度图的Mipmap来快速判断一个物体的包围盒是否可能被遮挡。\n\n### 12.2 细节层次 (Level of Detail, LOD)\n\nLOD技术的思想是，远处或不重要的物体不需要用高精度的模型来渲染。通过为同一个物体准备多个不同多边形数量的模型版本，并根据其与摄像机的距离或屏幕占比来切换使用哪个版本，可以显著降低GPU的几何处理负载。\n\n- 离散LOD: 准备多个独立的模型文件，在不同距离阈值进行切换。优点是简单可控，缺点是切换时可能会有明显的“跳变”感。\n- 连续LOD: 通过算法动态地简化或细分网格，实现平滑的细节过渡。实现复杂，但效果更好。\n\n### 12.3 渲染路径 (Rendering Paths)\n\n不同的渲染路径在处理多光源和不同材质时有各自的优缺点。\n\n#### 12.3.1 前向渲染 (Forward Rendering)\n\n这是最传统的渲染路径。对于每个需要被渲染的物体，依次计算场景中所有光源对它的光照影响，然后输出最终颜色。\n\n- 优点: 简单直观，原生支持透明物体和MSAA（多重采样抗锯齿）。\n- 缺点: 当场景中光源数量增加时，性能开销会急剧上升（每个物体 x 每个光源 的计算量）。\n\n#### 12.3.2 延迟渲染 (Deferred Shading)\n\n为了解决前向渲染在多光源下的性能问题，延迟渲染将渲染过程分为两个主要阶段：\n\n1. 几何遍 (Geometry Pass): 渲染所有物体，但不在这个阶段进行光照计算。而是将物体的各种信息（如位置、法线、颜色、材质属性等）输出到多个屏幕空间的缓冲区中，这些缓冲区合称为G-Buffer。\n2. 光照遍 (Lighting Pass): 渲染一个覆盖全屏的四边

形。对于每个像素，从G-Buffer中读取其几何和材质信息，然后计算所有光源对该像素的光照影响。\n\n- 优点: 光照计算的复杂度与屏幕上的像素数和光源数量成正比，与场景的几何复杂度无关。非常适合拥有大量动态光源的场景。\n- 缺点: G-Buffer需要大量的显存和带宽。原生不支持透明物体（因为G-Buffer只能存一层信息），并且与MSAA的兼容性不佳。\n\n#### 12.3.3 Forward+ (或称 Tiled Forward Shading)\n\nForward+ 结合了前向渲染和延迟渲染的优点。它通过将屏幕划分为一个个瓦片（Tile），并预先计算每个瓦片内有哪些光源会对其产生影响。\n\n在最终的着色阶段，对于每个片元，只需从其所属瓦片的光源列表中取出有效光源进行计算即可，而无需遍历所有光源。\n\n- 优点: 既能高效处理大量光源，又保留了前向渲染对透明物体和MSAA的良好支持。\n- 缺点: 需要GPU支持计算着色器（Compute Shader）来实现光源剔除，实现相对复杂。\n\n### 12.4 批处理 (Batching)\n\nCPU向GPU发送一次渲染命令（Draw Call）本身存在一定的开销。如果场景中有成千上万个小物体，频繁的Draw Call会成为CPU的性能瓶颈。批处理的目标就是减少Draw Call的数量。\n\n- 静态合批 (Static Batching): 对于场景中所有静态的（不会移动、旋转、缩放）、且使用相同材质的物体，引擎可以在游戏开始前将它们的网格合并成一个大的网格，从而用一次Draw Call渲染所有这些物体。\n- 动态合批 (Dynamic Batching): 引擎在每一帧动态地将那些足够小的、使用

相同材质的动态物体合并在一起进行渲染。\n- GPU实例化 (GPU Instancing): 一种更现代、更高效的技术。它通过一次Draw Call告诉GPU渲染同一个网格多次，但每次使用不同的变换矩阵和属性。这非常适合渲染大量相似的物体，如树木、草地、人群等。\n\n## 参考文献\n\n本笔记的编写参考了大量的网络教程、课程讲义和技术博客。所有引用的信息源均已在研究过程中进行了验证和提取。以下是主要的信息来源列表，以供读者进一步学习和探索。\n\n- Vulkan教程：GPU架构和渲染管线\n- NVIDIA GPU风格架构\n- 实时渲染管线\n- 渲染性能优化之几种LOD层次细节总结\n- DXR光线追踪学习（1）API和渲染管线\n- 探讨Forward与DeFered和Forward+\n- OpenGL、Vulkan、DirectX、Metal对比分析\n- 3D物体渲染到2D屏幕的矩阵变换过程：模型变换、视图变换、投影变换\n- GPU渲染管线中的MVP变换矩阵\n- 二维光栅图形的扫描：直线的DDA、Bresenham算法\n- 一文读懂：计算机图形学基础之纹理映射\n- 现代计算机图形学（正交投影，透视投影，MVP变换）\n- 计算机图形学(八)-纹理映射、计算重心坐标、UV插值、双线性插值\n- zBuffer算法\n- 深度缓存器算法（Z-buffer）简介\n- 计算机图形学入门(GAMES101)数学基础\n- 现代计算机图形学基础 第1章[数学工具・上篇]\n- 《Games 101》总结与思考——Part 3:变换(二维与三维)(下)\n- 【个人笔记】速通计算机图形学基础\n- RGB、CMY、CMYK、YUV、HSV、HSI、LAB颜色空间详解\n- 图形变换与齐次坐标.ppt\n-

Unity中的光照模型概念辨析\n- Phong模型镜面反射在计算机图形学中的应用\n- 基于Cook-Torrance与GGX的微表面模型实现\n- 贝塞尔曲线与de Casteljau算法\n- GAMES202-高质量实时渲染笔记04-Environment Mapping\n- 计算机图形学课程资料：BRDF与反射模型\n- 微表面模型理论\n- GPU Gems：基于图像的光照\n- Phong与Blinn-Phong光照模型讲义\n- GAMES101：Shadow Mapping笔记\n- 阴影技术：Shadow Volume\n- 高级软阴影映射技术\n- 非均匀有理B样条(NURBS)\n- 细分曲面算法\n- Cohen-Sutherland裁剪算法\n- 交互式鲁棒网格布尔运算\n- 几何相交算法\n- Bezier曲线算法\n- 3D表面细分方法文档\n- 双向反射分布函数(BRDF)\n